

THE AUTONOMOUS DATA WATCHER

Final Year Project Master Blueprint

V1 Implementation Plan + V2 Scalable Architecture

Core End-to-End Product Flow



Project in one sentence

A web platform that turns ordinary Excel/CSV business data into automatic anomaly detection, plain-English explanations, email alerts, and a simple visual dashboard.

Prepared as a consolidated explanation of the approved project direction

How to Read This Blueprint

This document is designed so you can explain the project to a professor, teammate, interviewer, or non-technical person without needing a second document. It separates the product into a realistic V1 that will be built and demonstrated, and a larger V2 architecture that shows how the same idea can grow into a production-grade SaaS platform.

Section	What it answers
1. Project Overview	What are we building and why does it matter?
2. Problem & Value	What real problem does it solve?
3. Scope Strategy	What is V1, and what is intentionally deferred?
4. User Flow	Exactly what happens from upload to alert?
5. Architecture	How do the frontend, API, analytics, database and email fit together?
6. Analytics	How are trend changes and anomalies detected?
7. Product Experience	What screens and outputs will the user see?
8. Backend & Data	What endpoints and data structures are needed?
9. Build Roadmap	In what order will the project be implemented?
10. Testing & Demo	How will we prove it works?
11. V2 Roadmap	How can the system scale after the final-year project?
12. Presentation Guide	How to explain the project in 30 seconds, 2 minutes, and a viva.

Core decision

Build a complete, polished V1 that proves the autonomous monitoring idea. Do not spend the project schedule on authentication, multi-tenancy, or production-scale infrastructure before the core pipeline is finished.

1. Project Overview

1.1 The idea

Many small and medium businesses still operate from spreadsheets. A manager may record sales, waste, inventory, margin, order volume, staff hours, returns, or other operational metrics in Excel, but the spreadsheet itself does not actively warn the manager when something unusual happens.

The Autonomous Data Watcher changes the spreadsheet from a passive record into an active monitoring system. The manager uploads a CSV or Excel file, tells the system which column represents time and which columns should be monitored, and the system performs statistical checks automatically. When a meaningful abnormality appears, the system explains it in simple language and can send an email alert.

Core product flow

Excel/CSV -> Column Mapping -> Analytics -> Anomaly Detection -> Explanation -> Email Alert + Dashboard

1.2 What makes the project interesting

- It is not only a CRUD website; the core value is analytical decision support.
- It combines full-stack development, data engineering, statistics, visualization, automation and optional AI-assisted wording.
- The user does not need to understand Z-scores, standard deviation or Python. The system translates technical findings into a short business explanation.
- The architecture is deliberately small enough to finish, but structured so it can later become a multi-user SaaS application.

1.3 Primary V1 outcome

By the end of V1, a non-technical person should be able to open the application, upload a supported spreadsheet, map the date and metric columns, run analysis, view flagged anomalies on a chart, read a plain-English summary, and receive an email when the configured threshold is breached - without needing help from the developer.

2. Problem, Users & Value Proposition

2.1 Problem statement

Business operators often suffer from dashboard fatigue: they have data, but not enough time to manually inspect every row, compare each period, calculate changes, or recognize when a value is statistically unusual. Problems may therefore be noticed only after waste, lost sales, stock issues, or margin pressure have already caused damage.

2.2 Target users

User type	Example monitored metrics	Why the watcher helps
Retail / store manager	Sales, returns, waste, stock levels	Flags sudden changes without manual spreadsheet review.
Franchise operator	Daily revenue, order volume, labor hours	Surfaces branches or periods behaving differently from normal.
Local service business	Bookings, cancellations, revenue	Warns about abnormal drops or spikes.
Boutique / specialty store	Margin, units sold, inventory	Provides quick operational alerts from existing spreadsheets.
Student/demo user	Any time-series numeric dataset	Shows the system is domain-flexible rather

User type	Example monitored metrics	Why the watcher helps
		than hardcoded to one industry.

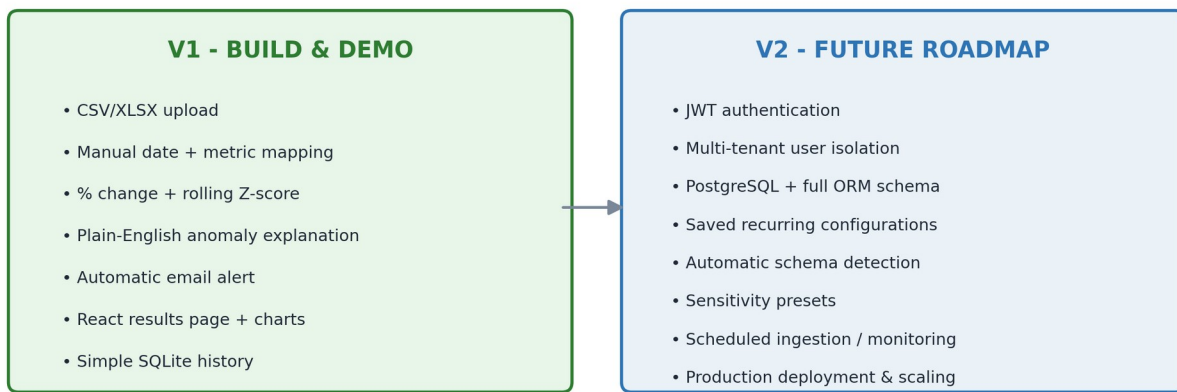
2.3 Value proposition

Value proposition

Replace passive spreadsheet checking with low-effort, autonomous monitoring: compute trend changes, identify anomalies, explain them in plain English, and proactively alert the user when something deserves attention.

3. Scope Strategy: What We Build vs. What We Design

Scope Strategy: Implement V1, Design for V2



A finished V1 proves the core idea. V2 proves the system can grow.

3.1 V1 - required implementation

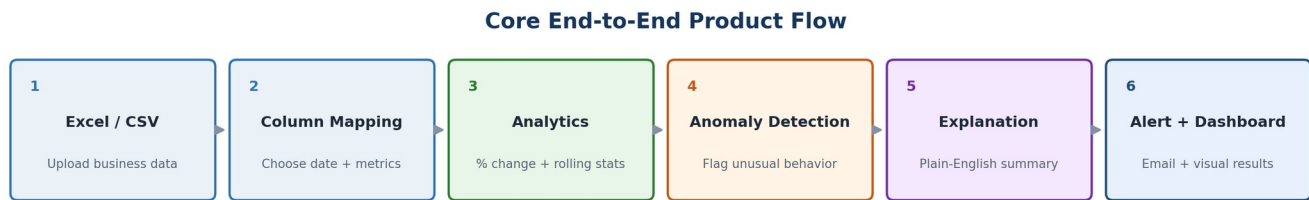
- CSV and XLSX upload using pandas/openpyxl.
- Manual column mapping: one date/time column and one or more numeric metric columns.
- Data validation and cleaning for missing/invalid values.
- Period-over-period percentage change.
- Rolling mean, rolling standard deviation, and rolling Z-score anomaly detection.
- Plain-English summary generated from computed results.
- Automatic one-way email alert when at least one metric breaches the selected rule.
- React results page with KPI cards, anomaly table, and a simple time-series chart.
- Lightweight SQLite persistence for analysis history and previous summaries.
- Sample datasets and a repeatable three-minute demonstration.

3.2 V2 - explicitly deferred

- Full JWT authentication, password reset and session management.
- Multi-tenant organizations and per-user data isolation.
- Production PostgreSQL schema with Users, Projects, Datasets and AlertLogs.
- Saved recurring upload configurations and scheduled monitoring.
- Automatic schema/column inference with no manual mapping.
- Advanced threshold presets, custom notification preferences and richer dashboards.
- Cloud deployment, horizontal scaling, queues/background workers and production observability.

Deferring these features is a project-management decision, not a technical limitation. The final report can show the full V2 architecture to demonstrate system-design awareness while the live demo focuses on a complete, reliable V1.

4. End-to-End User & Data Flow



1. Upload

User selects or drags a .csv/.xlsx file. Backend validates file type and reads the header/preview.

2. Guided mapping

UI asks: Which column is the date? Which numeric columns should be monitored? This keeps V1 reliable across different spreadsheet schemas.

3. Data preparation

Backend normalizes dates, converts metrics to numeric values, handles missing values, sorts by date, and checks that enough historical rows exist.

4. Statistical analysis

For each metric, calculate period-over-period percentage change plus rolling mean, rolling standard deviation and Z-score.

5. Anomaly decision

Flag observations that cross the statistical threshold and optionally combine this with a meaningful percentage-change rule.

6. Explanation

Convert the flagged numeric result into 2-3 sentences a manager can understand in seconds.

7. Dispatch

If an anomaly exists, automatically send an email. If nothing breaches the rule, do not send an unnecessary alert.

8. Results

Show summary, current values, flagged metrics, chart markers and optionally save the run to SQLite for history.

4.1 Example from raw spreadsheet to alert

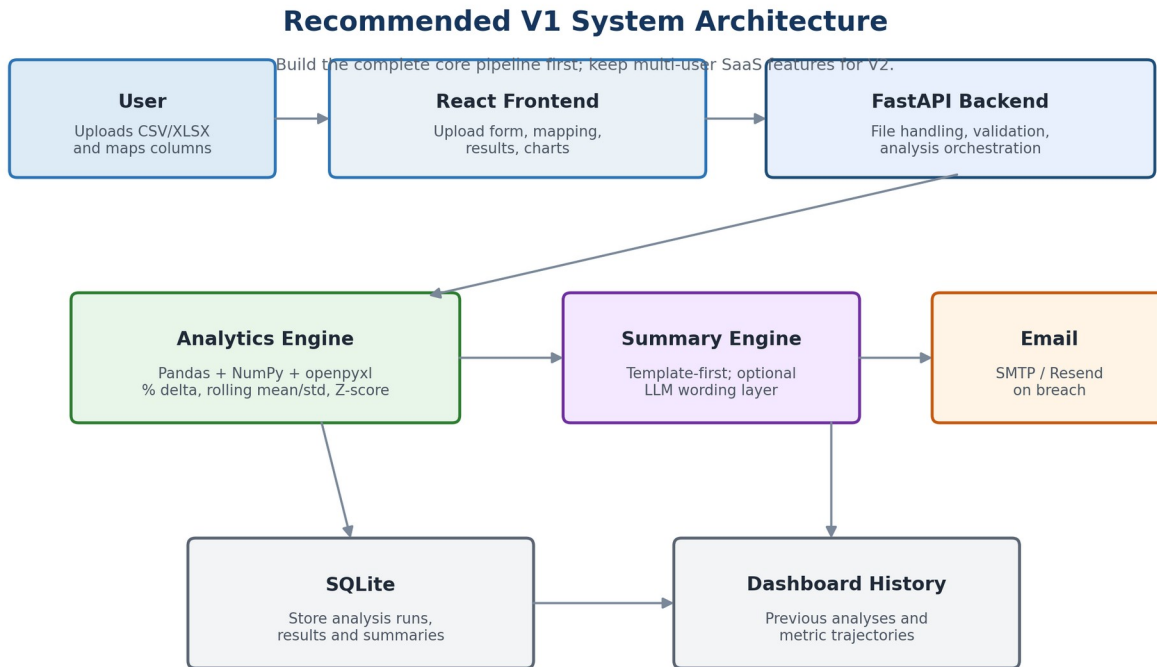
Date	Sales	Waste	Inventory
2026-08-01	5200	120	840
2026-08-02	5100	115	825
2026-08-03	5050	118	810
...			
2026-08-18	3100	390	760

Suppose the waste value on August 18 is far above its recent historical pattern. The analytics engine might calculate a large positive period-over-period change and a Z-score above the threshold.

Example user-facing output

WARNING: Waste increased sharply compared with the previous period and is significantly above its recent historical range. Review inventory handling, spoilage, or operational changes for this period.

5. Recommended V1 System Architecture



5.1 Technology stack

Layer	Recommended technology	Responsibility
Frontend	React + Tailwind CSS	Upload UI, mapping controls, results cards, anomaly table, charts and history view.
Backend API	FastAPI (Python)	Multipart upload, validation, API routing, analytics orchestration and response formatting.
Analytics	pandas + NumPy + openpyxl	Spreadsheet parsing, cleaning, rolling statistics, deltas and Z-scores.
Visualization	Recharts	Line charts with anomaly markers and metric comparison.
Persistence	SQLite for V1	Store analysis runs, filename/date, metric results and generated summaries.
Email	SMTP or Resend/SendGrid	Send transactional alert only when a breach occurs.
Narrative layer	Templates first; optional LLM	Turn computed statistics into concise

Layer	Recommended technology	Responsibility
		business language. Analytics remains deterministic.

5.2 Important architecture rule

Statistics decide; language explains

The anomaly decision should come from deterministic calculations, not from an LLM. If an LLM is used, it receives already-computed facts and only improves wording. This makes the project easier to test, explain, and defend in a viva.

6. Analytics & Anomaly Detection

6.1 Method A - period-over-period percentage change

```
delta_pct = (current_value - previous_value) / previous_value
```

This answers a simple business question: how much did the current value change compared with the immediately previous period? It is easy to explain and useful for sudden jumps or drops.

6.2 Method B - rolling Z-score

```
rolling_mean = mean(previous N periods)
rolling_std = standard_deviation(previous N periods)
Z = (current_value - rolling_mean) / rolling_std
flag anomaly when |Z| >= 2.0 # sensible V1 default
```

The Z-score asks whether the current observation is unusual relative to its own recent history. A rolling window of roughly 14-30 periods can be used depending on the dataset. This avoids comparing a metric only with one previous row.

6.3 Why use both

Situation	% change	Rolling Z-score	Interpretation
Large one-period jump	Strong signal	May also flag	Immediate shock.
Gradual trend drift	May show repeated modest changes	Can identify movement away from historical range	Slow deterioration or growth.
Naturally volatile metric	Can overreact	Adjusts for historical variability	Z-score provides context.
Stable metric with small abnormal move	May look numerically small	Can still be statistically unusual	Useful for sensitive metrics.

6.4 Data-quality rules

- Sort records by the mapped date column before calculating rolling statistics.
- Reject or report datasets where the mapped metric cannot be converted to numeric values.
- Handle missing values explicitly rather than allowing silent calculation errors.

- Avoid division by zero in percentage-change calculations.
- Require enough historical observations before evaluating a rolling Z-score.
- Return transparent intermediate values (mean, standard deviation, Z-score, delta) so every flag can be explained.

7. Plain-English Explanation & Alert Logic

7.1 Summary generation strategy

V1 should be able to generate useful summaries without an AI dependency. A template engine can translate computed facts into sentences. An optional LLM call can later make the wording more natural, but it should never invent metrics or decide whether an anomaly exists.

```
Input facts:
metric = "Waste"
current = 390
previous = 118
delta_pct = +230.5%
z_score = 2.7
```

```
Template output:
"Waste increased 230.5% from the previous period and is 2.7 standard deviations above its recent average. This is an unusual spike and should be reviewed."
```

7.2 Alert decision

- No anomaly -> show normal results on the dashboard; do not send email.
- One or more anomalies -> create a concise summary, save the run, and trigger one email containing all important breaches.
- Avoid one email per row or per metric; group alerts so the system does not create notification fatigue.
- Include the metric name, date/period, current value, comparison, severity evidence and a link or instruction to review the dashboard.

7.3 Example email

Subject: Autonomous Data Watcher - anomaly detected

Waste: 390, up sharply from the previous period and 2.7 standard deviations above the recent average. Sales also decreased materially. Review the latest operational data in the dashboard.

8. Product Experience: Screens to Build

Screen	Main elements	Done when...
1. Upload	File picker / drag-drop, supported formats, sample dataset option	A CSV/XLSX can be selected and previewed.
2. Mapping	Date dropdown, multi-select metric dropdown, analyze button	Different column names work without code changes.
3. Results	Executive summary, KPI cards, anomaly table, one line chart per selected metric	A non-technical user understands what happened.
4. History	Recent analyses, date, filename, number of flags, summary	A previous run can be opened again.

Screen	Main elements	Done when...
5. Error states	Invalid file, missing date, nonnumeric metric, insufficient rows	The user gets a useful message instead of a crash.

8.1 Results page layout

1. Top: short executive summary - maximum 2-3 sentences.
2. KPI row: metrics analyzed, anomalies detected, latest period, alert status.
3. Anomaly table: metric, date, value, percentage change, Z-score, status.
4. Chart area: time-series line with anomalous points visually highlighted.
5. Bottom: technical details / calculation values for transparency and debugging.

UI principle

The interface should make the insight obvious before showing the math. The mathematics is available for transparency, but the first thing the user sees is what changed and why it deserves attention.

9. Backend API & Data Design

9.1 Minimal V1 endpoints

Method & endpoint	Purpose	Typical output
POST /api/upload	Accept file, validate, parse header and preview rows	filename, columns, row_count, preview
POST /api/analyze	Accept file/reference + mapping, run analytics and summary	metrics, anomalies, summary, chart arrays
GET /api/history	Return recent saved analysis runs	run id, date, filename, alert count
GET /api/history/{id}	Return a saved result for review	full prior analysis payload

9.2 Simple V1 persistence

Use a lightweight SQLite database rather than the full multi-tenant schema. One normalized implementation can use an AnalysisRun table plus an AnalysisMetric table, or a single table with a JSON result payload. The goal is to prove persistence and history without spending weeks on account infrastructure.

Entity	Suggested fields
AnalysisRun	id, filename, uploaded_at, row_count, date_column, summary, alert_sent
AnalysisMetric	id, run_id, metric_name, latest_value, previous_value, delta_pct, z_score, is_anomaly

9.3 Example analysis response

```
{
  "summary": "Waste increased sharply and is outside its recent normal range.",
  "alert_sent": true,
  "metrics": [
    {"name": "Waste", "value": 390, "delta_pct": 230.5, "z_score": 2.7, "is_anomaly": true}
  ]
}
```

10. Implementation Roadmap

Phase	Build	Verification / definition of done
1	Standalone analytics engine	Given a CSV + mapping, Python outputs deltas, rolling statistics, Z-scores and anomaly flags as JSON.
2	Summary generator	Computed JSON becomes 2-3 accurate, readable sentences without changing the anomaly decision.
3	FastAPI wrapper	Upload and analyze work through Postman/curl and return the same correct JSON.
4	Email dispatch	A deliberately anomalous sample dataset triggers a real test email; normal data does not.
5	React frontend	A non-technical user can upload, map and analyze from the browser.
6	Visualization + SQLite	Charts show anomalies and recent runs persist in history.
7	Polish & documentation	Sample data, README, screenshots, architecture diagram, test evidence and a cold 3-minute demo are ready.

10.1 Development priority rule

Protect the core differentiator

If a feature does not directly improve upload -> analysis -> anomaly -> explanation -> alert -> dashboard, defer it until the core flow is working end-to-end.

10.2 Recommended implementation order inside the analytics engine

- Load CSV/XLSX and return columns.
- Validate user mapping.
- Normalize date column and sort records.
- Convert selected metrics to numeric values.
- Calculate period-over-period delta.
- Calculate rolling mean and standard deviation.

12. Calculate Z-score and flag anomalies.
13. Format a stable JSON result.
14. Add unit tests before connecting frontend/email.

11. Testing, Evaluation & Demo Plan

11.1 Functional test cases

Test	Expected result
Valid CSV upload	Columns and preview returned successfully.
Valid XLSX upload	Spreadsheet parsed with the same mapping workflow.
Invalid extension	Clear validation error; server does not crash.
Wrong date mapping	Useful error explaining the date cannot be parsed.
Text metric selected	Useful numeric conversion error.
Normal dataset	No anomaly email; dashboard shows normal status.
Injected spike	Correct metric/date flagged; chart marker and summary match calculation.
Injected drop	Negative anomaly correctly detected and explained.
Zero previous value	No divide-by-zero crash; percentage change handled explicitly.
Insufficient history	Rolling Z-score withheld or marked unavailable until enough periods exist.

11.2 Technical evaluation

- Unit-test formulas against manually calculated examples.
- Test multiple column names and datasets to prove the system is not hardcoded.
- Compare flagged points with known injected anomalies.
- Record API response time on small/medium spreadsheet samples.
- Verify that every generated narrative is grounded in values returned by the analytics engine.
- Verify alert suppression: normal datasets must not generate unnecessary emails.

11.3 Three-minute demo script

15. Open the application and explain the business problem in one sentence.
16. Upload a sample spreadsheet.
17. Map Date, Sales and Waste.
18. Click Analyze.
19. Show the executive summary and highlighted chart anomaly.
20. Open the automatic email alert.
21. Show the saved history entry.
22. Finish by showing the V2 architecture and explain that authentication/multi-tenancy are intentionally future work.

12. Risks & How We Control Them

Risk	Why it matters	Mitigation
Scope creep	Auth, SaaS infrastructure and advanced UI could delay the core product.	Freeze V1 scope until the complete end-to-end flow works.
Messy spreadsheets	Real files have missing values and inconsistent date formats.	Validate mappings, normalize inputs, and provide clear error messages.
False positives	A fixed statistical threshold may flag expected volatility.	Use rolling context, expose Z-score, document threshold assumptions, and add tunable sensitivity in V2.
LLM hallucination	A language model could invent causes or numbers.	Keep statistics deterministic; restrict LLM to wording from supplied facts.
Email reliability	SMTP/API credentials can fail during demo.	Test in advance and keep dashboard output as the primary result; email is an additional dispatch channel.
Too little history	Rolling statistics are not meaningful with very few rows.	Require a minimum history length and explain when a metric cannot yet be scored.

13. V2 Scalable SaaS Architecture

Once V1 is stable, the original full blueprint becomes the growth plan. The product can be transformed from a single-user analytical tool into a multi-tenant web platform while keeping the same analytics core.

13.1 V2 additions

- JWT authentication, password hashing, session/token lifecycle and account management.
- Organizations/companies with tenant-aware data isolation.
- PostgreSQL + SQLAlchemy models such as Users, Projects, Datasets and AlertLogs.
- Saved mapping configurations for recurring weekly/monthly uploads.
- Scheduled ingestion and background analysis.
- Automatic schema recognition with confidence checks.
- Threshold/sensitivity presets and per-metric rules.
- Multiple notification channels and richer alert history.
- Production deployment, logging, monitoring, queues and scaling.

13.2 Why the V1 architecture supports V2

FastAPI already exposes the analysis engine through APIs, React is separated from the backend, and analytics is implemented as a reusable Python module. Therefore, authentication, PostgreSQL, background workers and multi-tenant access can be added around the core rather than forcing the analytics logic to be rewritten.

14. How to Explain the Project to Anyone

14.1 30-second explanation

Elevator pitch

Businesses often keep important operational data in Excel but do not have time to constantly analyze it. My project is

a web application where a user uploads a spreadsheet and selects the metrics they care about. The backend automatically calculates trend changes and statistical anomalies, explains unusual behavior in plain English, displays it on a dashboard, and sends an email alert when something needs attention.

14.2 Two-minute technical explanation

The frontend is built in React and sends uploaded CSV/XLSX files plus user-selected column mappings to a FastAPI backend. pandas and NumPy clean the data and calculate both period-over-period percentage change and a rolling Z-score. The statistical engine determines whether an observation is anomalous. A summary layer then converts those facts into short business language, and an email service dispatches an alert when a breach occurs. Results are visualized through Recharts and a lightweight SQLite database stores analysis history. The V1 is intentionally single-user so the final-year product is complete; the full design documents a future move to JWT authentication, PostgreSQL and multi-tenant SaaS architecture.

14.3 What to emphasize in a viva

- Problem-first design: existing spreadsheet users should not have to adopt a complex BI tool.
- Manual mapping is deliberate in V1 because it provides reliability across arbitrary column names without pretending schema detection is solved.
- The system uses two complementary anomaly signals, not just a hardcoded threshold.
- The language layer does not decide truth; the analytics layer does.
- V1/V2 separation demonstrates scope management and architecture planning.
- Every anomaly can be explained using transparent intermediate calculations.

14.4 Resume / portfolio version

Portfolio bullet

Built an autonomous business-data monitoring web app using React, FastAPI, pandas/NumPy and SQLite that ingests Excel/CSV data, detects time-series anomalies using period-over-period change and rolling Z-scores, generates plain-English explanations, visualizes flagged points, and dispatches threshold-based email alerts.

15. Final Definition of Success

The project is successful when the complete user journey works reliably, not when the longest possible feature list exists.

Success criterion	Target
Usability	A non-technical user completes upload -> mapping -> analysis -> result without developer help.
Generality	Works on multiple CSV/XLSX schemas through manual mapping; no dataset-specific hardcoding.
Analytical correctness	Percentage change and Z-score outputs match test calculations.
Explainability	Every flag includes the values that caused it and a concise understandable narrative.
Automation	A breach automatically triggers one grouped email alert.
Visualization	User can see the metric trajectory and anomaly location.
Persistence	Recent analysis results are retrievable from SQLite.

Success criterion	Target
Demo readiness	The full value proposition can be demonstrated cold in under three minutes.
Architecture maturity	Report clearly distinguishes implemented V1 from designed V2 production architecture.

Final project principle

Finish the intelligence pipeline first. A complete autonomous watcher with a simple interface is stronger than a beautiful multi-user shell with unfinished analytics.

Appendix A - Recommended Folder Structure

```

autonomous-data-watcher/
├── backend/
│   ├── app/main.py
│   ├── app/api/
│   ├── app/services/analytics.py
│   ├── app/services/summary.py
│   ├── app/services/emailer.py
│   ├── app/db.py
│   └── tests/
├── frontend/
│   ├── src/components/
│   ├── src/pages/
│   ├── src/services/api.js
│   └── src/App.jsx
├── sample_data/
├── docs/
├── README.md
└── .env.example

```

Appendix B - Source Alignment

This master blueprint consolidates the two supplied project documents: the original full technical architecture and the intentionally scoped V1 build plan. The full architecture contributes the end-to-end workflow, technology choices, API/database direction and statistical core; the V1 build plan contributes the scope discipline, minimal implementation sequence and explicit V2 deferrals. The small enhancements in this master version - React as the default frontend, lightweight SQLite history, template-first narrative generation, testing strategy and presentation guidance - reflect the agreed final-year implementation direction.